

1 What it Does

Sorts a list of integers or real numbers.

2 How it Works:

We pass through the list from left to right. If we encounter an entry which is smaller than some of its predecessors then we need to move it as far left as is appropriate. We shift as many elements to the right as needed to make room for it and then insert it (hence the name) in the proper position.

3 Visualization

Consider the following list:

4	3	1	8	3
---	---	---	---	---

We don't bother to look at $A[0]=4$ because nothing to the left could be larger, because there's nothing to the left of it.

Thus we first look at $A[1]=3$ and note that there is one larger element to the left:

4	3	1	8	3
---	---	---	---	---

We shift it to the right and insert the 3 where it belongs:

3	4	1	8	3
---	---	---	---	---

We then look at $A[2]=1$ and note that there are two larger elements to the left:

3	4	1	8	3
---	---	---	---	---

We shift those to the right and insert the 1 where it belongs:

1	3	4	8	3
---	---	---	---	---

We next look at $A[3]=8$ and note that there are no larger elements to the left:

1	3	4	8	3
---	---	---	---	---

We do nothing. Again officially the pseudocode will actually shift nothing and then assign $A[3]=8$, which is a bit wasteful but there we go.

We then look at $A[4]=3$ and note that there are two larger elements to the left:

1	3	4	8	3
---	---	---	---	---

We shift those to the right and insert the 3 where it belongs:

1	3	3	4	8
---	---	---	---	---

Then we are done.

```
#include <stdio.h>

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; ++i)
        printf("%d ", arr[i]);
    printf("\n");
}

int main()
{
    int arr[] = { 12, 11, 13, 5, 6 };
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);
    printArray(arr, n);

    return 0;
}
```